

ShopFlow BD

F-commerce Order & Delivery Management Platform

A multi-tenant SaaS platform that turns Facebook and Instagram order chatter into a controlled order-to-cash pipeline: confirmation, inventory, courier booking, delivery tracking, COD reconciliation, returns, and true net profit.

Document version	1.0
Status	Draft for review
Date	18 September 2026
Author	Jahid H. R.
Product type	Multi-tenant B2B SaaS (web)
Target market	Bangladesh — Facebook/Instagram commerce sellers

Contents

- 1. Executive Summary
- 2. Problem Statement
- 3. Goals & Non-Goals
- 4. Success Metrics
- 5. Personas
- 6. Roles & Permissions
- 7. Functional Requirements
- 8. Order Lifecycle
- 9. Data Model
- 10. API Design
- 11. Frontend Specification
- 12. Technical Architecture
- 13. Release Plan
- 14. Risks & Mitigations
- 15. Open Questions
- Appendix A — Profit Calculation
- Appendix B — Glossary

1. Executive Summary

Bangladesh has one of the world's largest social-commerce markets by seller count. Tens of thousands of sellers run full businesses inside Facebook Page inboxes, Instagram DMs, and WhatsApp threads. They have real revenue but no operating system: orders live in chat scrollback, customer addresses are copy-pasted into courier websites, Cash-on-Delivery (COD) money is reconciled by hand against courier statements, and profit is guessed at the end of the month.

ShopFlow BD is a multi-tenant SaaS platform that becomes the single operational record for these sellers. A seller enters or imports an order once; the platform carries it through confirmation, inventory reservation, courier booking, delivery tracking, COD collection, reconciliation, returns, and finally into a profit report that is accurate to the taka.

The MVP is deliberately scoped to the **order-to-cash loop**, which is where sellers lose the most money and time. Everything else — Messenger automation, multi-warehouse, accounting exports — is sequenced behind that loop.

Product thesis: *The core pain in Bangladeshi F-commerce is not order-taking. It is that nobody knows which orders actually turned into collected cash. ShopFlow BD wins by owning COD reconciliation and true per-order profit, not by being another order form.*

2. Problem Statement

2.1 The current seller workflow

A typical seller processing 40 orders/day does the following, manually, for every order:

1. Reads the order details out of a Messenger thread or a phone call.
2. Copies name, phone, and address into a notebook, an Excel sheet, or a Facebook post comment.
3. Calls the customer to confirm — often twice, because fake and prank orders are common.
4. Looks up the delivery charge for the customer's district from memory.
5. Opens the courier's own web panel and re-types the same customer data to book a parcel.
6. Writes the courier's consignment ID next to the order in the notebook.
7. Answers "where is my parcel?" messages by opening the courier panel and searching.
8. At month-end, receives a courier settlement statement and tries to match it line-by-line against the notebook.
9. Guesses profit, because product cost, delivery cost, return shipping cost, and ad spend were never recorded against individual orders.

2.2 Quantified pain

Pain point	Impact
Double data entry	Customer data is typed 2–3 times per order (sheet, courier panel, receipt). ~3–4 min/order of pure re-typing.
Fake & unconfirmed orders	20–40% of social-commerce orders are not genuine. Shipping them unconfirmed means paying return freight both ways.
High return rate	Industry COD return rates run 15–30%. Each return costs delivery charge + return charge + restocking labour, and is rarely attributed back to the order.
COD reconciliation blindness	Courier statements arrive as PDFs/Excel with consignment IDs, not order numbers. Sellers cannot tell which COD is still uncollected, so courier shortfalls go unnoticed.
No true profit figure	Revenue is visible; COGS, delivery cost, return loss, discounts, and ad spend are not. Sellers routinely run unprofitable SKUs for months.
Inventory drift	Stock sold across Page, DM, and phone is not deducted anywhere. Overselling causes cancellations, which damage the Page's reputation.
No staff accountability	When an employee handles orders there is no audit trail of who changed what, and no way to limit what they can see.

2.3 Why now

- Courier APIs (Pathao, Steadfast, RedX, eCourier, Paperfly) are now publicly documented and accessible to small merchants.
- bKash/Nagad merchant APIs make partial advance payments practical, which is the main defence against fake orders.
- Existing tools are either too heavy (Shopify + plugins, priced and designed for card-paying Western storefronts) or too light (a spreadsheet template). The COD-native middle is unoccupied.

3. Goals & Non-Goals

3.1 Product goals

Goal	
G1	Reduce per-order handling time from ~6 minutes to under 60 seconds for a confirmed, courier-booked order.
G2	Give every seller a COD ledger that always answers: how much cash is in transit, how much is overdue, how much is reconciled.
G3	Produce per-order and per-product net profit that a seller trusts enough to make buying decisions from.
G4	Make courier booking a one-click action with zero re-typing, across at least three couriers.
G5	Cut return losses by surfacing risky customers and orders before they ship.

3.2 Explicit non-goals for v1

Not in v1	Rationale
Public storefront	We are not building a customer-facing shop. Sellers keep selling on Facebook/Instagram; we manage what happens after the order.
Messenger chatbot	No automated reply bot in v1. Order intake is manual entry, quick-entry form, and a public order form link.
Full accounting	No double-entry ledger, VAT returns, or payroll. We produce exportable data for the seller's accountant.
Multi-warehouse / multi-branch	Single stock pool per store in v1.
Marketplace integrations	No Daraz/Shopee sync in v1.
Native mobile apps	Responsive web only; the dashboard must be fully usable on a phone.

4. Success Metrics

Area	Metric	Target
Activation	% of registered stores that create ≥ 1 product AND ship ≥ 1 order within 7 days	$\geq 45\%$
Core engagement	Weekly active stores that process ≥ 10 orders/week	$\geq 60\%$ of paying stores
Time-to-ship	Median seconds from order creation to courier booking	$< 90s$
Courier automation rate	% of shipments booked via API vs. manual entry	$\geq 80\%$
Reconciliation coverage	% of delivered orders with a matched COD payment record	$\geq 95\%$
Retention	Month-3 logo retention of paying stores	$\geq 70\%$
Reliability	Courier-sync job success rate; API p95 latency	$\geq 99\%$ / $< 400ms$
Support load	Support tickets per 100 active stores per week	< 8

5. Personas

5.1 Rumana — Solo seller (primary)

Sells women's clothing from home via a Facebook Page with 40k followers. 25–50 orders/day in season. Uses an Android phone as her primary computer, a laptop in the evening. Not technically trained; abandons any tool that requires more than one screen of learning. Her biggest fear is shipping to a fake order and eating the return freight.

Needs: fast order entry on a phone, one-tap courier booking, a clear "who owes me money" screen.

5.2 Tanvir — Growing store owner with staff (primary)

Runs an electronics accessories business, 150–300 orders/day, 3 employees, a small rented stockroom. Already uses a spreadsheet and a WhatsApp group for coordination. Needs delegation without handing over the whole business.

Needs: role-based staff accounts, an audit trail, product-level profit to decide what to restock, bulk courier booking.

5.3 Sadia — Order-processing staff (secondary)

Employed by Tanvir. Spends her whole shift confirming orders by phone and booking parcels. Should see orders and customers but not revenue, profit, cost prices, or settings.

Needs: a fast work queue, call-outcome logging, no ambiguity about what to do next.

5.4 Karim — End customer (tertiary, unauthenticated)

Bought over Messenger. Wants to know where his parcel is without messaging the Page. Gets a public tracking link.

Needs: a mobile-friendly status page keyed by tracking code + phone, no login.

6. Roles & Permissions

Role	Scope
Owner	Full control of the store: billing, settings, staff management, all financial data, deletion rights. One per store (transferable).
Manager	Everything operational plus financial reports and cost prices. Cannot manage billing, delete the store, or change courier credentials.
Order Staff	Create/edit/confirm orders, manage customers, book shipments, log call outcomes. Cost prices, profit, analytics, and settings are hidden.
Delivery Staff	Read-only on orders; can update shipment status and record returns. No customer list export.
Accountant	Read-only on orders and shipments; full access to payments, COD reconciliation, expenses, and reports. Cannot modify orders.
Platform Admin	ShopFlow internal. Manages plans, courier integrations, and support impersonation (audit-logged, consent-gated).

6.1 Permission matrix

The matrix below defines the v1 authorization surface. Every DRF viewset enforces it via a permission class plus a store-scoped queryset.

Capability	Owner	Manager	Order	Deliv.	Acct.
View orders	Yes	Yes	Yes	Yes	Yes
Create / edit orders	Yes	Yes	Yes	—	—
Confirm / cancel orders	Yes	Yes	Yes	—	—
Book shipments	Yes	Yes	Yes	—	—
Update shipment status	Yes	Yes	Yes	Yes	—
Record returns	Yes	Yes	Yes	Yes	—

Capability	Owner	Manager	Order	Deliv.	Acct.
View / edit products	Yes	Yes	View	—	—
See cost price & margin	Yes	Yes	—	—	Yes
Manage customers	Yes	Yes	Yes	View	View
Export customer list	Yes	Yes	—	—	—
Record payments	Yes	Yes	—	—	Yes
COD reconciliation	Yes	Yes	—	—	Yes
Manage expenses	Yes	Yes	—	—	Yes
View analytics & profit	Yes	Yes	—	—	Yes
Manage staff & roles	Yes	—	—	—	—
Courier credentials	Yes	—	—	—	—
Store settings	Yes	Yes	—	—	—
Billing & plan	Yes	—	—	—	—
Delete store	Yes	—	—	—	—

7. Functional Requirements

Requirements are tagged **P0** (MVP, must ship), **P1** (fast-follow, within 2 releases), **P2** (roadmap).

7.1 Authentication & onboarding

ID	Pri	Requirement
FR-1.1	P0	Email + password registration with email verification. Phone number captured at signup (BD format, +880 normalisation).
FR-1.2	P0	JWT authentication: short-lived access token (15 min), rotating refresh token (7 days), refresh-token blacklist on logout.
FR-1.3	P0	Store creation wizard on first login: store name, slug, logo, contact phone, district, default courier, default delivery charges.
FR-1.4	P0	Password reset by email link; rate-limited.
FR-1.5	P0	Staff invitation by email with a role; invite expires in 72h. Invited user sets their own password.
FR-1.6	P1	Two-factor authentication via SMS OTP for Owner and Manager roles.
FR-1.7	P1	Login via phone + OTP as an alternative to email (many sellers do not use email daily).
FR-1.8	P2	Google OAuth sign-in.

7.2 Store & settings

ID	Pri	Requirement
FR-2.1	P0	Store profile: name, logo, contact details, address, business type, timezone (Asia/Dhaka), currency (BDT, fixed in v1).
FR-2.2	P0	Delivery charge rules: a default inside-Dhaka charge, outside-Dhaka charge, and optional per-district overrides. Free-delivery threshold.
FR-2.3	P0	Order number format configuration (prefix + sequence, e.g. RB-1042), unique per store.
FR-2.4	P0	Invoice settings: header, footer note, terms text, whether to show cost prices (never) and discounts.
FR-2.5	P0	Courier credential storage per store, encrypted at rest (Fernet with a key from the secrets manager, never plaintext in DB or logs).
FR-2.6	P1	SMS template management for order confirmation, shipped, and delivered notifications.
FR-2.7	P1	Store-level fraud thresholds: auto-flag a customer after N returns or a return rate above X%.

7.3 Product & inventory

ID	Pri	Requirement
FR-3.1	P0	Product CRUD: name, SKU (auto-generated if blank, unique per store), category, description, images (up to 6), selling price, cost price, weight in grams.
FR-3.2	P0	Variants on up to two axes (e.g. Size × Colour) with per-variant SKU, price override, cost override, and independent stock.
FR-3.3	P0	Stock tracking with three counters: <code>on_hand</code> , <code>reserved</code> (in confirmed-but-unshipped orders), and derived <code>available = on_hand - reserved</code> .
FR-3.4	P0	Stock ledger: every movement is an immutable row (order reservation, shipment, return restock, manual adjustment with a reason). Current stock is always reconstructable.
FR-3.5	P0	Low-stock threshold per product with a dashboard alert list.
FR-3.6	P0	CSV import and export of products, with a dry-run validation report before commit.
FR-3.7	P1	Barcode/QR generation per variant for stockroom picking.
FR-3.8	P1	Product bundles (a sellable item composed of several stocked items).
FR-3.9	P2	Purchase orders and supplier records to drive cost-price history (weighted average cost).

7.4 Customer management

ID	Pri	Requirement
FR-4.1	P0	Customer record keyed by normalised phone number, unique per store. Name, alternate phone, Facebook profile URL/name, district, upazila, full address, notes.
FR-4.2	P0	Automatic customer create-or-attach during order entry by phone lookup. Typing a known phone autofills the address.
FR-4.3	P0	Per-customer computed metrics: total orders, delivered, returned, cancelled, lifetime value, return rate, last order date.
FR-4.4	P0	Customer risk badge derived from return rate and return count: Good / Watch / High Risk / Blacklisted. Blacklisted customers raise a hard warning on order creation.
FR-4.5	P0	Multiple saved addresses per customer; one default.
FR-4.6	P1	Cross-store anonymised fraud signal: a national return-rate indicator computed from aggregate platform data, surfaced without exposing other stores' data.
FR-4.7	P1	Customer merge tool for duplicates created by phone-format variance.

7.5 Order management

ID	Pri	Requirement
FR-5.1	P0	Fast order entry form: phone → autofill customer → add line items with live stock check → auto delivery charge by district → discount → advance paid → COD amount computed and displayed live.
FR-5.2	P0	Order fields: source (Messenger / Instagram / WhatsApp / Phone / Public Form / Manual), items with quantity and unit price snapshot, subtotal, discount, delivery charge, advance paid, COD amount, internal note, customer note.
FR-5.3	P0	Unit price and product name are snapshotted onto the order item at creation; later product edits never mutate historical orders.
FR-5.4	P0	Order status lifecycle with enforced legal transitions (see §8). Every transition writes an OrderStatusHistory row with actor, timestamp, note.
FR-5.5	P0	Confirmation workflow: log call outcome (Confirmed / No answer / Asked to call later / Cancelled / Fake). Attempt counter. Confirming reserves stock.
FR-5.6	P0	Order editing is allowed only in Pending/Confirmed. From Processing onward, items are frozen; only notes and the shipping address may change (address change is audit-logged).
FR-5.7	P0	Cancellation with a mandatory reason from a controlled list; releases reserved stock.
FR-5.8	P0	Order list with server-side filters (status, courier, date range, district, source, staff, payment state), free-text search on order number / customer name / phone, and pagination.
FR-5.9	P0	Bulk actions: confirm, mark ready-to-ship, assign courier, print invoices — over a filtered selection.
FR-5.10	P0	Printable A5 invoice and a thermal-printer-friendly parcel label (PDF).
FR-5.11	P0	Duplicate-order detection: warn when the same phone has an open order with overlapping items in the last 24h.
FR-5.12	P1	Public order form per store (/s/<s lug>/order) that creates a Pending order — for sellers who prefer posting a link over collecting details in chat.
FR-5.13	P1	Partial fulfilment: split one order into two shipments.
FR-5.14	P1	Order exchange flow (return item A, ship item B) as a linked order pair.
FR-5.15	P2	Messenger integration for semi-automatic order capture from a conversation.

7.6 Courier & shipment

ID	Pri	Requirement
FR-6.1	P0	Courier registry: platform-level courier definitions (Pathao, Steadfast, RedX, eCourier, Paperfly, plus a generic "Manual/Local" courier) and per-store enablement with credentials.
FR-6.2	P0	A courier adapter abstraction : one internal interface (<code>create_parcel</code> , <code>track</code> , <code>cancel</code> , <code>price_quote</code> , <code>parse_webhook</code>) with a concrete adapter per courier. Adding a courier must require no changes to order or shipment code.
FR-6.3	P0	One-click shipment creation from an order: builds the payload from stored data, calls the courier API, stores <code>consignment_id</code> and tracking URL, moves the order to Shipped.
FR-6.4	P0	Manual shipment mode: record a consignment ID entered by hand for couriers/local riders without an API.
FR-6.5	P0	Periodic Celery beat task polls tracking status for all in-transit shipments; interval scales with shipment age. Every observed status writes a <code>DeliveryStatusHistory</code> row.
FR-6.6	P0	Courier status values are mapped to our canonical order statuses through an explicit per-courier mapping table, so a courier renaming a status cannot corrupt our lifecycle.
FR-6.7	P0	Shipment cost captured per shipment (quoted vs. actual charged), because courier invoices differ from quotes.
FR-6.8	P0	Webhook endpoints per courier where supported, with signature verification and idempotent handling; polling remains the fallback.
FR-6.9	P1	Bulk booking: select N orders, book them all, show a per-order success/failure report.
FR-6.10	P1	Courier performance comparison: delivery success rate, average delivery days, return rate, and average cost per courier per district.
FR-6.11	P1	Smart courier suggestion for a destination district based on that store's own historical performance.

7.7 Payments & COD reconciliation

This is the platform's differentiating surface and is treated as a first-class module, not a field on the order.

ID	Pri	Requirement
FR-7.1	P0	Payment records against an order, each with method (COD / bKash / Nagad / Rocket / Bank / Cash), amount, direction (in/out), reference, received-at, and recorded-by.
FR-7.2	P0	Advance payment at order time reduces the COD amount; the order shows Paid / Partially Paid / Unpaid.
FR-7.3	P0	COD lifecycle per shipment: Pending → Collected by courier → Settled to seller, with the settlement date and the courier's deducted charge.
FR-7.4	P0	Courier settlement import: upload the courier's CSV/Excel statement; the system matches rows to shipments by consignment ID, shows matched / unmatched / amount-mismatch buckets, and commits only on confirmation.
FR-7.5	P0	COD ledger screen: total in transit, collected but unsettled, settled this period, overdue (delivered more than N days ago with no settlement), and shortfalls (settled amount < expected COD).
FR-7.6	P1	bKash/Nagad merchant API verification of an advance payment by transaction ID.
FR-7.7	P1	Automated settlement fetch via courier API where a settlement endpoint exists.

7.8 Returns

ID	Pri	Requirement
FR-8.1	P0	Return record linked to an order and shipment: type (Full / Partial / Exchange), reason from a controlled list (Customer refused, Wrong item, Damaged, Size issue, Not available at delivery, Fake order, Other), and per-item return quantities.
FR-8.2	P0	Return resolution actions: restock to inventory, or mark damaged/written-off (which keeps it out of sellable stock but records the loss).
FR-8.3	P0	Return cost accounting: forward delivery charge + return charge are both booked against the order so the loss is visible per order.
FR-8.4	P0	Refund handling for orders that were pre-paid: create an outgoing payment record.
FR-8.5	P0	Returning an order updates the customer's return metrics and may auto-escalate their risk badge.
FR-8.6	P1	Return analytics: return rate by product, variant, district, courier, and source channel — to find the actual cause.

7.9 Expenses

ID	Pri	Requirement
FR-9.1	P0	Expense records: date, category (Ad spend, Packaging, Salary, Rent, Courier charge, Product purchase, Utilities, Other), amount, note, optional attachment.
FR-9.2	P0	Expenses feed the net-profit calculation for their period.
FR-9.3	P1	Recurring expense templates (monthly rent, salaries) auto-created by a scheduled task.
FR-9.4	P1	Attribute ad spend to a campaign and compare against orders whose source matches, giving an approximate ROAS.

7.10 Dashboard & analytics

ID	Pri	Requirement
FR-10.1	P0	Dashboard KPI row: today's orders, pending, delivered today, returned today; today's sales, delivery cost, gross profit, net profit. Each with a comparison against the same figure yesterday.
FR-10.2	P0	Dashboard widgets: sales trend (last 30 days), order-status funnel, top 5 products by revenue, low-stock alerts, orders needing confirmation, overdue COD.
FR-10.3	P0	Sales report over an arbitrary date range, grouped by day/week/month, with order count, gross sales, discounts, delivery revenue, delivery cost, COGS, returns loss, and net profit.
FR-10.4	P0	Product performance report: units sold, revenue, COGS, gross margin, return rate — sortable, so a seller can find loss-making SKUs.
FR-10.5	P0	Courier performance report and district performance report.
FR-10.6	P0	CSV/Excel export on every report.
FR-10.7	P1	Staff performance report: orders handled, confirmation rate, average handling time.
FR-10.8	P1	Scheduled daily summary by email/SMS at a store-configured hour.
FR-10.9	P2	Cohort retention of customers and repeat-purchase rate.

7.11 Notifications

ID	Pri	Requirement
FR-11.1	P0	In-app notification centre: new public-form order, low stock, COD overdue, courier booking failure, return recorded.
FR-11.2	P0	Transactional email to sellers for account and billing events.
FR-11.3	P1	SMS to the customer on Confirmed, Shipped, and Delivered, using a local gateway; per-store toggle and template, with SMS credits metered.
FR-11.4	P2	Web push for order events.

7.12 Public customer tracking

ID	Pri	Requirement
FR-12.1	P0	Public page at /track accepting tracking code or (order number + phone). No authentication.
FR-12.2	P0	Shows a status timeline, courier name, and expected delivery window. Exposes no other customer's data, no cost or profit figures, and is rate-limited against enumeration.

7.13 Billing & plans

ID	Pri	Requirement
FR-13.1	P1	Plans: Free (up to 50 orders/month, 1 user), Starter, Growth, Business — differentiated by monthly order volume, staff seats, courier integrations, and SMS credits.
FR-13.2	P1	Usage metering on order count per billing period, with soft warnings at 80% and a block on creation past the hard cap.
FR-13.3	P1	Manual bank/bKash payment activation by Platform Admin in the first release; automated recurring billing afterwards.

8. Order Lifecycle

8.1 Canonical statuses

Status	Meaning
Pending	Order captured, not yet verified with the customer. No stock reserved.
Confirmed	Customer verified the order by phone/chat. Stock is reserved at this moment.
Processing	Being picked and packed in the stockroom.
Ready to Ship	Packed, labelled, awaiting courier pickup.
Shipped	Handed to the courier; a consignment ID exists.
Out for Delivery	Rider is attempting delivery (driven by courier tracking).
Delivered	Customer received the parcel. Revenue is recognised; COD becomes collectable.
Cancelled	Terminated before shipping. Reserved stock is released.
Returned	Parcel came back, fully or partly. Return costs are booked; stock may be restocked.
On Hold	Blocked — customer unreachable, stock-out, or an address problem. Parked without losing the order.

8.2 Legal transitions

From	Allowed next states
Pending	Confirmed, Cancelled, On Hold
Confirmed	Processing, Ready to Ship, Cancelled, On Hold
Processing	Ready to Ship, Cancelled, On Hold
Ready to Ship	Shipped, Cancelled, On Hold
Shipped	Out for Delivery, Delivered, Returned
Out for Delivery	Delivered, Returned, Shipped (re-attempt)
Delivered	Returned (post-delivery return window)
On Hold	Pending, Confirmed, Cancelled
Cancelled	terminal
Returned	terminal

Any transition not listed is rejected by the API with 409 `Conflict` and a machine-readable reason. A backward or corrective move (for example `Delivered` → `Returned`) is permitted where listed and always requires a note.

8.3 Side effects

Transition	Side effect
→ Confirmed	Reserve stock for every line item; fail the transition if stock is insufficient.
→ Shipped	Convert reservation into an outbound stock movement; decrement on_hand; set COD status to Pending.
→ Delivered	Recognise revenue for reports; mark COD collectable; update customer success metrics.
→ Cancelled	Release reservations; void the COD expectation; require a reason.
→ Returned	Create/complete a Return record; book return costs; restock or write off; update customer risk metrics.

9. Data Model

9.1 Entity list

All business tables carry `store_id` (except `User`, `Store`, `Courier`, `Plan`), plus `created_at`, `updated_at`, and soft-delete `deleted_at` where records must survive for audit.

Entity	Fields & notes
User	Platform account. Email (unique), phone, name, <code>is_active</code> , <code>last_login</code> . Authentication only — no business data.
Store	A tenant. Name, slug (unique), owner FK to User, logo, contact phone, address, district, timezone, plan FK, order-number prefix and sequence, <code>is_active</code> .
StoreMembership	Join of User ↔ Store with role. Enables one user in several stores and is the source of truth for authorization.
StoreSettings	One-to-one with Store. Delivery charge defaults, per-district overrides (JSON), free-delivery threshold, invoice text, SMS toggles and templates, fraud thresholds, default courier FK.
Category	Per-store product category. Name, slug, parent (self-FK, one level in v1).
Product	Store FK, name, slug, SKU, category FK, description, <code>selling_price</code> , <code>cost_price</code> , <code>weight_grams</code> , <code>has_variants</code> , <code>low_stock_threshold</code> , <code>is_active</code> .
ProductImage	Product FK, image, alt text, sort order, <code>is_primary</code> .
ProductVariant	Product FK, SKU (unique per store), <code>option1_name/value</code> , <code>option2_name/value</code> , <code>price_override</code> , <code>cost_override</code> , weight override, <code>is_active</code> .
StockItem	One row per stockable unit (product without variants, or variant). <code>on_hand</code> , <code>reserved</code> , <code>low_stock_threshold</code> . Unique on (product, variant).
StockMovement	Immutable ledger. StockItem FK, type (purchase / order_reserve / reserve_release / sale / return_restock / write_off / adjustment), quantity (signed), reference (order/return/adjustment), reason, actor FK, <code>created_at</code> .
Customer	Store FK, phone (normalised, unique per store), name, <code>alt_phone</code> , <code>facebook_url</code> , <code>facebook_name</code> , notes, <code>risk_level</code> , <code>is_blacklisted</code> , cached metrics (<code>total_orders</code> , <code>delivered_count</code> , <code>returned_count</code> , <code>cancelled_count</code> , <code>lifetime_value</code> , <code>last_order_at</code>).
CustomerAddress	Customer FK, label, division, district, upazila/thana, area, <code>address_line</code> , <code>is_default</code> .
Order	Store FK, <code>order_number</code> (unique per store), customer FK, shipping address snapshot (denormalised text + district), status, source, subtotal, <code>discount_amount</code> , <code>delivery_charge</code> , <code>total_amount</code> , <code>advance_paid</code> , <code>cod_amount</code> , <code>payment_status</code> , <code>internal_note</code> , <code>customer_note</code> , confirmation attempt count, last call outcome, <code>created_by</code> FK, <code>confirmed_at</code> , <code>shipped_at</code> , <code>delivered_at</code> , <code>cancelled_at</code> , <code>cancel_reason</code> .

Entity	Fields & notes
OrderItem	Order FK, product FK (protected), variant FK, product_name snapshot, sku snapshot, unit_price snapshot, unit_cost snapshot, quantity, line_total.
OrderStatusHistory	Order FK, from_status, to_status, note, actor FK, created_at. Append-only.
Courier	Platform-level. Name, code, logo, adapter key, supports_api, supports_webhook, is_active.
StoreCourier	Store FK, Courier FK, encrypted credentials (JSON), is_enabled, is_default, per-courier config (pickup store id, etc.).
Shipment	Order FK (one active per order), StoreCourier FK, consignment_id, tracking_code, tracking_url, booking_mode (api/manual), quoted_cost, actual_cost, cod_amount, cod_status, cod_collected_at, cod_settled_at, settlement_reference, raw booking request/response (JSON, for support), current_courier_status, booked_at, delivered_at.
DeliveryStatusHistory	Shipment FK, courier raw status, mapped canonical status, courier note, location, observed_at, source (poll/webhook/manual). Append-only.
Payment	Store FK, Order FK, method, direction (in/out), amount, reference, note, received_at, recorded_by FK, is_verified.
CourierSettlement	Store FK, StoreCourier FK, statement reference, period start/end, uploaded file, total_amount, matched_count, unmatched_count, status (draft/committed), created_by FK.
SettlementLine	CourierSettlement FK, raw row (JSON), consignment_id, amount, deducted_charge, matched Shipment FK (nullable), match_status (matched / unmatched / amount_mismatch).
Return	Store FK, Order FK, Shipment FK, type, reason, status (initiated / received / resolved), return_charge, refund_amount, resolution_note, created_by FK, received_at, resolved_at.
ReturnItem	Return FK, OrderItem FK, quantity, condition (sellable / damaged), restocked (bool).
Expense	Store FK, date, category, amount, note, attachment, created_by FK.
Notification	Store FK, User FK (nullable = store-wide), type, title, body, payload (JSON), is_read, created_at.
AuditLog	Store FK, actor FK, action, target model + id, changes (JSON diff), IP, user agent, created_at. Append-only, written for all financially or legally sensitive mutations.
Plan / Subscription	Plan: name, price, order cap, seat cap, feature flags. Subscription: store FK, plan FK, period, status, usage counters.

9.2 Key relationships

Relationship	Rule
Store → Products / Customers / Orders / Expenses / Staff	One-to-many, and the tenancy boundary for every query.
Order → Customer	Many-to-one, PROTECT on delete (a customer with orders cannot be hard-deleted).
Order → OrderItems	One-to-many, cascade. Items hold price/cost snapshots.
Order → Shipment	One active shipment per order in v1 (model allows many for partial fulfilment in P1).
Order → Payments	One-to-many. Sum of inbound minus outbound payments drives payment_status.
Order → Return	One-to-many (partial returns), each with its own ReturnItems.
Product → Variants → StockItem	StockItem is attached to the variant when variants exist, otherwise to the product.
Shipment → DeliveryStatusHistory	One-to-many, append-only tracking trail.
CourierSettlement → SettlementLines → Shipment	Reconciliation join: each statement line resolves to at most one shipment.

9.3 Data integrity rules

Data integrity rule
Every list query is filtered by the caller's store. Enforced by a base queryset mixin, not by remembering to add a filter.
Unique constraints scoped to store: (store, order_number), (store, customer.phone), (store, product.sku), (store, variant.sku).
Money is DecimalField(max_digits=12, decimal_places=2). Floats are never used for money anywhere in the stack.
Stock changes and status transitions run inside a DB transaction with select_for_update on the affected StockItem rows, so concurrent confirmations cannot oversell.
Order numbers are allocated by an atomic per-store counter, not by COUNT(*) + 1.
History tables (OrderStatusHistory, DeliveryStatusHistory, StockMovement, AuditLog) are append-only: no update or delete path exists in the API.

10. API Design

10.1 Conventions

Convention	Rule
Base path	/api/ — the version is in the URL from day one.
Auth	Authorization: Bearer <access token>. Store context is derived from the token's membership; a X-Store-Id header selects among multiple memberships.
Pagination	Cursor pagination on high-volume list endpoints (orders, customers), page-number elsewhere. Default page size 25, max 100.
Filtering	django-filter query params. Multi-value via repeated params (?status=shipped&status=delivered).
Errors	Consistent envelope: {"error": {"code": "INVALID_TRANSITION", "message": "...", "details": {...}}}. Codes are stable and documented.
Idempotency	Mutating endpoints that touch money or couriers accept an Idempotency-Key header; a repeated key returns the original result instead of acting twice.
Rate limits	Per-user and per-store throttles; tighter limits on public tracking and the public order form.
Docs	OpenAPI 3 schema generated by drf-spectacular, served at /api/schema/ with Swagger UI at /api/docs/.

10.2 Endpoint surface

Authenti cation	Purpose
POST /auth/register/	Create user account
POST /auth/login/	Obtain access + refresh tokens
POST /auth/refresh/	Rotate access token
POST /auth/logout/	Blacklist refresh token
POST /auth/password/reset/	Request reset email
GET /auth/me/	Current user + memberships

Stores & staff	Purpose
GET POST /stores/	List own stores / create store
GET PATCH /stores/{id}/	Store profile
GET PATCH /stores/{id}/settings/	Delivery charges, invoice, SMS
GET POST /stores/{id}/staff/	List / invite staff
PATCH DELETE /stores/{id}/staff/{uid}/	Change role / revoke access

Catalog		Purpose
GET POST	/categories/	Categories
GET POST	/products/	List (filter, search) / create
GET PATCH DEL	/products/{id}/	Product detail
POST	/products/{id}/variants/	Add variant
POST	/products/import/	CSV import (dry-run + commit)
GET	/products/export/	CSV export
GET	/stock/	Stock levels, low-stock filter
POST	/stock/adjust/	Manual adjustment with reason
GET	/stock/movements/	Immutable stock ledger

Customers		Purpose
GET POST	/customers/	List / create
GET PATCH	/customers/{id}/	Detail + metrics + risk
GET	/customers/lookup/?phone=	Autofill during order entry
GET	/customers/{id}/orders/	Order history
POST	/customers/{id}/blacklist/	Blacklist / un-blacklist

Orders		Purpose
GET POST	/orders/	List (filters, search) / create
GET PATCH	/orders/{id}/	Detail / edit (pre-ship only)
PATCH	/orders/{id}/status/	Transition status (state machine)
POST	/orders/{id}/confirm/	Log call outcome + confirm
POST	/orders/{id}/cancel/	Cancel with reason
GET	/orders/{id}/history/	Status timeline
GET	/orders/{id}/invoice/	Invoice PDF
GET	/orders/{id}/label/	Parcel label PDF
POST	/orders/bulk-status/	Bulk transition
POST	/orders/check-duplicate/	Duplicate-order warning

Couriers & shipments		Purpose
GET	/couriers/	Available couriers

Couriers & shipments	Purpose	
GET POST	/store-couriers/	Enable courier, store credentials
POST	/shipments/	Book parcel (API or manual)
GET	/shipments/	List with COD / status filters
GET	/shipments/{id}/	Detail + tracking history
POST	/shipments/{id}/sync/	Force tracking refresh
POST	/shipments/{id}/cancel/	Cancel with courier
POST	/shipments/bulk-book/	Bulk booking + per-order report
POST	/shipments/quote/	Delivery price quote
POST	/webhooks/courier/{code}/	Courier status webhook (signed)

Payments & COD	Purpose	
GET POST	/payments/	Payment records
GET	/payments/cod-ledger/	In transit / unsettled / overdue
POST	/settlements/	Upload courier statement
GET	/settlements/{id}/preview/	Matched / unmatched / mismatch
POST	/settlements/{id}/commit/	Commit reconciliation

Returns & expenses	Purpose	
GET POST	/returns/	List / create return
PATCH	/returns/{id}/	Update / resolve
POST	/returns/{id}/restock/	Restock or write off items
GET POST	/expenses/	Expense records

Analytics	Purpose	
GET	/analytics/dashboard/	KPI row + widgets (cached)
GET	/analytics/sales/	Sales & profit by period
GET	/analytics/products/	Product performance
GET	/analytics/couriers/	Courier performance
GET	/analytics/districts/	District performance
GET	/analytics/returns/	Return analysis
GET	/analytics/export/	CSV / Excel export

Public (unauthenticated)		Purpose
GET	/public/track/?code=&phone=	Customer order tracking
POST	/public/stores/{slug}/orders/	Public order form submission

10.3 Representative request / response

```
POST /api/v1/orders/
Authorization: Bearer <token>
Idempotency-Key: 8f14e45f-ea0b-4f1b-9f2a-2c0b7d5e1a33
```

```
{
  "customer": { "phone": "01712345678", "name": "Karim Ahmed" },
  "shipping_address": {
    "district": "Dhaka", "thana": "Dhanmondi",
    "address_line": "House 12, Road 4, Dhanmondi"
  },
  "source": "messenger",
  "items": [
    { "product": 41, "variant": 118, "quantity": 2 },
    { "product": 57, "quantity": 1 }
  ],
  "discount_amount": "50.00",
  "advance_paid": "100.00",
  "customer_note": "Please deliver after 5 PM"
}
```

```
201 Created
{
  "id": 9042,
  "order_number": "RB-1042",
  "status": "pending",
  "customer": { "id": 733, "name": "Karim Ahmed", "phone": "+8801712345678",
    "risk_level": "good", "previous_orders": 3 },
  "items": [ ... ],
  "subtotal": "1850.00",
  "discount_amount": "50.00",
  "delivery_charge": "60.00",
  "total_amount": "1860.00",
  "advance_paid": "100.00",
  "cod_amount": "1760.00",
  "payment_status": "partially_paid",
  "created_at": "2026-09-18T11:04:22+06:00"
}
```

```
PATCH /api/v1/orders/9042/status/
{ "status": "confirmed", "note": "Confirmed by phone, 2nd attempt" }
```

```
200 OK
{ "id": 9042, "status": "confirmed",
  "confirmed_at": "2026-09-18T11:31:08+06:00",
  "stock_reserved": true }
```

```
409 Conflict
{ "error": { "code": "INSUFFICIENT_STOCK",
  "message": "Cannot confirm: not enough stock.",
  "details": { "variant": 118, "requested": 2, "available": 1 } } }
```

11. Frontend Specification

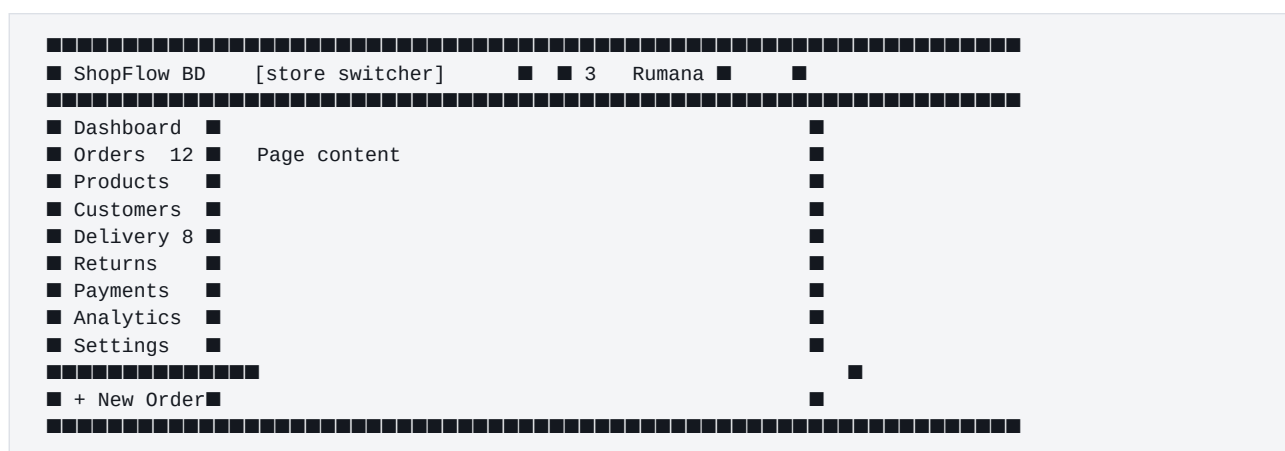
11.1 Stack

Layer	Choice
Framework	React 18 with Vite, JavaScript (ES2022+) throughout. No TypeScript build step; correctness at the API boundary is enforced by runtime schema validation instead (see Forms and Testing).
Routing	React Router v6 with nested layouts and route-level code splitting.
Server state	TanStack Query — caching, background refetch, optimistic updates on status changes.
Client state	Zustand for auth/session and UI state. No global store for server data.
Forms	React Hook Form + Zod for form validation. Zod schemas are also applied to every API response, so a malformed or unexpected payload fails loudly at the boundary rather than propagating through the UI — this is the primary substitute for compile-time types.
Styling	Tailwind CSS with a small design-token layer; shadcn/ui/Radix primitives for accessible components.
Charts	Recharts for dashboard and analytics visuals.
Tables	TanStack Table with server-driven sorting, filtering, and pagination.
i18n	react-i18next with English and Bangla from v1; Bangla numerals in currency display are configurable.
Testing	Vitest + React Testing Library for units; Playwright for the critical order-to-shipment path. Because there is no compile-time type check, frontend coverage on money formatting, status-transition guards and API-response parsing is treated as mandatory rather than optional.

11.2 Route map

Route	Contents
/login, /register, /forgot-password, /invite/:token	Public auth screens.
/onboarding	First-run store-creation wizard.
/dashboard	KPI row, sales trend, funnel, action lists (needs confirmation, low stock, overdue COD).
/orders	Filterable order table, saved views per status, bulk actions, quick-entry drawer.
/orders/new	Full-page fast order-entry form.
/orders/:id	Order detail: items, customer panel with risk badge, status timeline, shipment card, payments, returns, notes, print actions.
/products, /products/new, /products/:id	Product list with stock column; detail with variants, images, stock ledger tab.
/customers, /customers/:id	Customer list with risk column; detail with order history and metrics.
/delivery	Shipment board grouped by shipment state; bulk booking; courier sync status.
/payments	COD ledger, settlement import wizard, payment records.
/returns	Return list and the return-processing flow.
/expenses	Expense entry and list.
/analytics	Sales, product, courier, district, and return reports with a date-range picker and export.
/settings	Store profile, delivery charges, couriers, staff & roles, invoice, notifications, billing.
/track	Public tracking page (outside the dashboard shell).

11.3 Shell layout



Sidebar items show live counts for work queues (orders awaiting confirmation, shipments needing attention). On screens narrower than 1024px the sidebar collapses to a bottom tab bar with the five most-used destinations, because a large share of sellers work from a phone.

11.4 UX requirements

Principle	Requirement
Keyboard-first order entry	The whole order form is completable without a mouse; Enter advances, a barcode/SKU field adds line items.
Sub-second perceived actions	Status changes and confirmations apply optimistically and roll back visibly on failure.
Never lose typed data	The order form autosaves a draft to local storage; a failed submit never clears the form.
Money is unambiguous	Every amount renders with the Tk symbol and two decimals; COD amount is visually dominant on order screens.
Risk is visible before shipping	The customer risk badge and prior-return count appear on the order form and the order detail header.
Empty states teach	Every empty list explains the next action and links to it.
Accessibility	WCAG 2.1 AA: focus rings, labelled inputs, 4.5:1 contrast, screen-reader-announced toasts.
Bangla-ready typography	A font stack that renders Bangla conjuncts correctly; no layout break on longer Bangla strings.

12. Technical Architecture

12.1 Stack decisions

Layer	Technology & rationale
Backend	Django 5.x + Django REST Framework
Database	PostgreSQL 15+ — chosen over MySQL/SQLite for JSONB (courier payloads, district overrides), partial indexes, and strong concurrent-transaction behaviour under <code>select_for_update</code> . SQLite is used only for local development bootstrap and unit tests.
Async tasks	Celery with Redis as broker and result backend; Celery Beat for scheduled jobs.
Cache / locks	Redis — dashboard metric caching, throttling counters, and distributed locks for courier sync.
Auth	<code>djangorestframework-simplejwt</code> with rotation and blacklisting.
Files	S3-compatible object storage (AWS S3 or DigitalOcean Spaces) via <code>django-storages</code> ; local filesystem in development.
API docs	<code>drf-spectacular</code> (OpenAPI 3).
Frontend	React 18 + JavaScript (ES2022+) + Vite, served as static assets behind a CDN. JSDoc annotations on shared API-client and money-handling modules give editor hints without a type-check step.
Serving	Gunicorn (or Uvicorn for ASGI) behind Nginx; static/media via CDN.
Monitoring	Sentry for errors, structured JSON logs shipped to a log store, Prometheus-compatible metrics, and uptime checks on the API and courier-sync jobs.
CI/CD	GitHub Actions: lint (ruff + mypy on the Django backend, ESLint on the React app), test with coverage gate, build, migrate, deploy. Docker + docker-compose for parity across environments.

12.2 Application layout

```

backend/
  ■■■■ config/                settings/{base,dev,prod}.py, urls, celery
  ■■■■ apps/
    ■ ■■■■ accounts/         User, auth, JWT views, invitations
    ■ ■■■■ stores/           Store, StoreMembership, StoreSettings, permissions
    ■ ■■■■ catalog/          Category, Product, Variant, StockItem, StockMovement
    ■ ■■■■ customers/        Customer, addresses, risk scoring
    ■ ■■■■ orders/           Order, OrderItem, status machine, numbering
    ■ ■■■■ couriers/         Courier registry + adapters/{pathao,steadfast,redx,...}
    ■ ■■■■ shipments/        Shipment, tracking history, sync tasks, webhooks
    ■ ■■■■ payments/         Payment, COD ledger, settlement import
    ■ ■■■■ returns/          Return, ReturnItem, restock logic
    ■ ■■■■ expenses/         Expense
    ■ ■■■■ analytics/        Report query services, aggregation tasks
    ■ ■■■■ notifications/    Notification, email/SMS dispatch
    ■ ■■■■ core/             Base models, mixins, pagination, exceptions, audit log
  ■■■■ tests/

```

Business logic lives in a `services.py` per app (`orders/services.py` owns `create_order`, `transition_status`, `cancel_order`). Views stay thin, serializers validate, services own invariants and transactions. This keeps the state machine testable without HTTP and callable from Celery tasks.

12.3 The courier adapter pattern

Every courier is behind one interface:

```

class CourierAdapter(Protocol):
    def price_quote(self, shipment: ShipmentDraft) -> Money: ...
    def create_parcel(self, shipment: Shipment) -> BookingResult: ...
    def track(self, consignment_id: str) -> list[TrackingEvent]: ...
    def cancel(self, consignment_id: str) -> bool: ...
    def parse_webhook(self, request) -> list[TrackingEvent]: ...
    STATUS_MAP: dict[str, OrderStatus]

```

Adapters are registered by the `Courier.adapter_key` and resolved at runtime. Each adapter is responsible for its own payload shape, auth, retry and status vocabulary; nothing outside `apps/couriers/adapters/` knows a courier's name. Every outbound call is wrapped with a timeout, bounded retries with exponential backoff, and a circuit breaker so one courier's outage cannot stall the booking queue. Raw request/response JSON is persisted on the shipment for support and dispute resolution.

12.4 Background jobs

Task	Cadence	Responsibility
sync_shipment_tracking	Every 15 min	Polls in-transit shipments in batches; interval widens for older shipments; writes tracking history and drives status transitions.
refresh_dashboard_metrics	Every 5 min	Recomputes and caches per-store KPI aggregates so the dashboard never runs heavy queries on request.
recompute_customer_metrics	Nightly	Rebuilds customer order/return metrics and risk levels from source data.
detect_overdue_cod	Daily	Flags shipments delivered more than N days ago without settlement and notifies the store.
send_daily_summary	Daily, per-store hour	Emails/SMSes yesterday's numbers.
expire_stale_pending_orders	Daily	Notifies (never auto-cancels) on Pending orders older than the store's threshold.
cleanup_and_archive	Weekly	Rotates raw courier payloads older than 90 days to cold storage.

12.5 Non-functional requirements

Requirement	Target
Performance	API p95 < 400ms for list endpoints at 100k orders per store. Dashboard loads from cache in < 1s. Order list queries are index-backed on (store, status, created_at).
Scale target	10,000 stores, 5M orders, 500 concurrent dashboard users on a single primary database with a read replica for analytics.
Availability	99.5% monthly for the API in v1. Courier outages degrade gracefully: booking queues and retries rather than failing the order.
Security	TLS everywhere; JWT rotation; courier credentials encrypted with envelope encryption; per-store data isolation verified by automated tests; OWASP Top 10 review before launch; secrets never in the repo; strict CORS allowlist; request-level audit logging on financial mutations.
Privacy	Customer phone numbers and addresses are personal data: access is role-gated, exports are audit-logged, and a store deletion request purges or anonymises customer records within 30 days. Cross-store fraud signals are aggregated and anonymised, never row-level.
Data durability	Automated daily encrypted database backups with 30-day retention, point-in-time recovery, and a quarterly restore drill.
Observability	Every courier call, Celery task, and status transition is logged with a correlation ID that ties an HTTP request to its downstream effects.
Testing	≥ 80% backend coverage overall and ≥ 95% on orders/services.py, the state machine, stock movements, and money arithmetic. Courier adapters are tested against recorded fixtures, never live APIs, in CI.
Frontend correctness	With no TypeScript layer, every API response is parsed through a Zod schema before it reaches a component, and money values are handled by a single shared helper module that never uses native float arithmetic. Both are covered by unit tests in CI.
Localisation	All user-facing strings externalised; English and Bangla shipped in v1; Asia/Dhaka timezone and BDT formatting are defaults.

13. Release Plan

13.1 Phase 1 — Foundation (Weeks 1–3)

Scope	Project scaffolding, Docker, CI. User/Store/Membership models, JWT auth, registration, store onboarding wizard, role permission framework, base queryset tenancy mixin, audit log. React app shell, routing, auth flow, layout.
Exit criteria	A user can register, create a store, invite staff with roles, and log in. Tenancy isolation is covered by tests.

13.2 Phase 2 — Catalog & Customers (Weeks 4–5)

Scope	Categories, products, variants, images, StockItem, StockMovement ledger, low-stock alerts, CSV import/export. Customers with phone normalisation, addresses, metrics.
Exit criteria	A seller can build a full catalog with variants and stock, and maintain a customer book.

13.3 Phase 3 — Orders (Weeks 6–9) — core release

Scope	Order model with snapshots, order numbering, the fast entry form, status state machine with side effects, status history, confirmation workflow with call outcomes, cancellation, stock reservation, duplicate detection, list filters, bulk actions, invoice and label PDFs.
Exit criteria	An order can travel Pending → Delivered with correct stock effects and a complete audit trail. This phase is the product's spine and gates everything after it.

13.4 Phase 4 — Delivery (Weeks 10–12)

Scope	Courier registry, credential encryption, two live adapters (Pathao, Steadfast) plus manual mode, one-click and bulk booking, tracking poll task, webhooks, status mapping, shipment costs, delivery board.
Exit criteria	An order is booked with a real courier in one click and its status updates itself without human action.

13.5 Phase 5 — Money (Weeks 13–15)

Scope	Payments, advance handling, COD lifecycle, COD ledger, courier settlement import and matching, returns with restock/write-off and cost booking, refunds, expenses.
Exit criteria	A seller can reconcile a real courier statement end-to-end and see which COD is missing.

13.6 Phase 6 — Insight & polish (Weeks 16–18)

Scope	Dashboard KPIs with caching, sales/product/courier/district/return reports, exports, notification centre, public tracking page, Bangla localisation, accessibility pass, performance tuning, seed data, documentation.
Exit criteria	Net profit is correct against a hand-checked month of test data, and the dashboard loads in under a second.

13.7 Phase 7 — Commercialisation (Weeks 19–20)

Scope	Plans, usage metering, order caps, manual subscription activation, SMS notifications with credits, third courier adapter, staff performance report.
Exit criteria	The platform can onboard and bill a paying customer.

14. Risks & Mitigations

Risk	Sev.	Mitigation
Courier APIs are unstable, undocumented, or change without notice	High	Every courier sits behind an adapter with recorded-fixture tests; manual booking mode is always available as a fallback; raw payloads are stored for debugging; a circuit breaker isolates one courier's failure.
COD settlement statements differ per courier and per month	High	The import is a wizard with explicit matched / unmatched / mismatch buckets and a human confirmation step. Nothing is auto-committed, so a format change degrades to manual matching rather than corrupting the ledger.
Sellers abandon the tool because data entry feels slower than their notebook	High	Order entry is the single most optimised screen: phone autofill, keyboard-only flow, live COD calculation, local-storage draft, sub-60-second target measured as a product metric.
Multi-tenant data leakage	Critical	Tenancy enforced by a base queryset mixin plus a permission class, with automated cross-tenant access tests in CI. No endpoint may filter by store ad hoc.
Oversell under concurrent confirmation	Medium	Reservation happens inside a transaction with <code>select_for_update</code> on stock rows; stock is derived from an immutable ledger so any drift is auditable.
Profit numbers are wrong and destroy trust	Critical	Cost is snapshotted per order item; delivery, return and refund costs are booked explicitly; the profit formula is documented in-app and validated against a hand-computed fixture month in CI.
Price sensitivity in the target market	Medium	A genuinely useful Free tier at 50 orders/month, taka-denominated pricing, and local payment methods (bKash/Nagad/bank) rather than card-only.
Scope creep into a full storefront or chatbot	Medium	Non-goals are written into §3.2 and the phase gates; anything outside the order-to-cash loop is deferred by default.

15. Open Questions

- Which two couriers ship first? Pathao and Steadfast are assumed based on market share, but adapter effort should be validated against real sandbox access before Phase 4 begins.
- Should the Free tier include courier API booking, or is that the primary upgrade trigger?
- Is the cross-store anonymised fraud signal legally and ethically acceptable in this market? It is the strongest possible differentiator and also the largest privacy question in the product. Requires a written policy before any implementation.
- Do sellers want inventory at all in v1, or is it a barrier during onboarding? A "stock tracking off" mode may be needed for sellers who buy per-order from a wholesaler.
- What is the right overdue-COD threshold by default — 7 days after delivery, or courier-specific?

Appendix A — Profit Calculation

The profit formula is stated explicitly because trust in this number is the product's core value proposition.

Gross Sales	= Σ order.subtotal	(delivered orders only)
Net Sales	= Gross Sales – Σ order.discount_amount	
Delivery Revenue	= Σ order.delivery_charge	(delivered orders)
COGS	= Σ (order_item.unit_cost × quantity)	(delivered orders)
Gross Profit	= Net Sales + Delivery Revenue – COGS – Delivery Cost	
Delivery Cost	= Σ shipment.actual_cost	(all shipments in period)
Return Loss	= Σ (forward delivery cost + return_charge + written-off item cost)	
Operating Expenses	= Σ expense.amount	(period)
Net Profit	= Gross Profit – Return Loss – Operating Expenses	

Revenue is recognised on **Delivered**, not on order creation — a Pending order in a COD market is not revenue. Delivery cost and return loss are counted in the period the shipment occurred, so a return recognised late correctly reduces that later period. Both a cash-basis and an order-basis view are offered in the analytics module, because sellers reason in cash while the data model reasons in orders.

Appendix B — Glossary

Term	Definition
F-commerce	Facebook-commerce: selling directly through Facebook/Instagram pages and chat rather than a website.
COD	Cash on Delivery. The dominant payment method; the courier collects cash and later settles it to the seller.
Consignment ID	The courier's own identifier for a parcel, and the key on which settlement statements are matched.
Settlement	The courier paying collected COD to the seller, less delivery and COD-handling charges.
Reserved stock	Units committed to confirmed-but-unshipped orders; not sellable, not yet dispatched.
Return rate	Returned orders ÷ shipped orders, over a period — the single most important health metric in this market.
Upazila / Thana	Sub-district administrative units, used in Bangladeshi addressing and courier coverage tables.